

RESEARCH ARTICLE

CovidBChain: Framework for access-control, authentication, and integrity of Covid-19 data

Vijayant Pawar | Shelly Sachdeva 

Department of Computer Science and Engineering, National Institute of Technology Delhi, Delhi, India

Correspondence

Shelly Sachdeva, Department of Computer Science and Engineering, National Institute of Technology Delhi, Delhi, India.
Email: shellysachdeva@nitdelhi.ac.in

Summary

In the Covid-19 pandemic, information about the medical equipment such as personal protective equipment, ventilators, testing kits, oxygen cylinders, ICU beds, and patient diagnostic status is a black box for the patients. This article proposes a blockchain-assisted Covid-19 big data chain (CovidBChain) framework to handle the Covid-19 data, which is of colossal size (volume), coming from different sources (variety) and generated at every time instance (velocity). CovidBChain is proposed to protect electronic health records and Covid-19 equipment's information from illegal modification. CovidBChain provides transparency, access control, and integrity to Covid-19 data. The status of critical equipment like ventilator, Covid-19 beds, oxygen cylinder, and ICU status each such operation is integrated into the CovidBChain as a transaction. A prototype has been simulated using Ganache, Metamask, InterPlanetary File System, and Reactjs. The comparative assessment using proof-of-work (PoW) and proof-of-authority (PoA) deduces that the upload and retrieval time in PoA is less than PoW, while the transaction cost is more in PoW. The overhead of message exchange communication is reduced by a factor of 4.3x in PoA as compared to the PoW approach. CovidBChain has been tested on the Ethereum official test network Ropsten for PoW and Goerli for PoA.

KEYWORDS

blockchain, Covid-19, healthcare

1 | INTRODUCTION

Industry 4.0 technologies such as the Internet of Things (IoT), big data, and artificial intelligence (AI) are already used in the diagnosis of diseases like glaucoma,¹ hyperactivity,² and Alzheimer's.³ Covid-19 is severely difficult and different to diagnose compared to other pandemics, and diseases humanity faced earlier. Due to its severeness and impact on society, it has become the profit-making source for the healthcare domain's evil entities. There have been instances where data manipulation is done for the following reasons; to show how well government handles the Covid-19 pandemic, to hide the insensibility of private hospitals, to hide the correct information about hospital amenities, and to get a more significant reimbursement. According to the report,⁴ Covid-19 data are altered due to political influence.⁵ Sixty-eight percent of Americans do not trust what their president says about Covid-19 figures. It shows a lack of trust among people towards accuracy, transparency, and predictions claimed by authorities for decision-making. Article⁶ says that if data integrity and accuracy are hampered, it is challenging for the researchers to make sense of it. Politicians aired their concern as the number of reported cases could be "restricted." The motive is to show others how well managed the city is during the pandemic, and everything is under control. According to Article 21 of the constitution,⁷ health protection and access to health care cannot be denied on financial grounds. The private sector's insensitivity is not hidden, so Covid-19 treatment cost needs to

be regularized. The tests and consumables required for the treatment must be transparent to the patients for the ethical functioning of the system. Kalpagge⁸ states treatment cost of the Covid-19 infection is more than the earnings of 94% of the population of India without including the Covid-19 tests cost.

Delhi government released an app to track the live status of hospital beds, ventilators, and ICU beds.⁹ However, there are already discrepancies between the app's data, bulletin data, and the existing facility in each hospital. Scientists and medical professionals are learning as going, which means changes to the protocol in collecting Covid-19 data must be made as new findings are revealed. Stevens et al.¹⁰ discuss flaws of the traditional healthcare system, such as discharging of corona positives may spread coronavirus admitting new patients without accurate disclosure about infection risks, and exposure of medical staff to Covid-19 due to false medical data. Pneumonia and Covid-19 have the same symptoms and similar effects on the respiratory and lungs. Some hospitals are taking advantage of this situation by calling the case of pneumonia a corona case. Texas's chief forensic pathologist in Reference 11 said hospital administrators might change patient discharge summaries to get a more significant reimbursement. Doctors are encouraged to cite a normal death as a Covid-19 death as money is a motivation.

Hence there is a dire need for a framework to combat above mentioned problems. The authors propose Covid-19 Big data Blockchain (CovidBChain) to handle the chaos caused by the Covid-19 pandemic.

1.1 | Brief overview of CovidBChain

Information regarding newly infected cases, deaths, discharge status, quarantine status, hospital amenities such as ICU, oxygen cylinders, ventilators, number of beds, and so forth, is enormous in volume and collected from various resources. So, hospitals cannot store every bit of Covid-19 information.

This article proposes a secure eHealth system called CovidBChain, which ensures the confidentiality of outsourced medical information and protects against illegal modification without introducing any centralized trusted entity. Instead of converging trust at a single point, trust is distributed among system stakeholders. CovidBChain uses permissioned Ethereum blockchain. It is a platform that shares information and performs the computation in a decentralized way. It plays a vital role in the decision-making process between different authoritative domains. Blockchain maintains each entity's privacy by generating a private/public key pair. It also supports non-repudiation using a digital signature and removing centralized storage dependency using decentralized/distributed storage.¹²

The key idea is to utilize the blockchain technique, which provides a tamper-proof and distributed way to conduct transactions without a central authority. In CovidBChain, the metadata about a patient's medical information and hospital amenities status is integrated into a blockchain transaction only after storing actual (patient and hospital amenities) data in the InterPlanetary File System (IPFS). The security of CovidBChain is ensured even if the doctor colludes with the third-party storage service.

In the Covid-19 pandemic's adverse situation, privacy, security, transparency, integrity, and access-control of data are urgent. Evil entities of the society always want to gain benefit from this precarious situation by manipulating multimedia data related to Covid-19 pandemic. The healthcare sector needs to be more secure and transparent to fight against malicious actors in the healthcare system. Thus, a blockchain-enabled solution to counter the evils present in this work. The main benefit of using blockchain in health care is stakeholder anonymity, conditional privacy preservation,¹³ and security¹⁴ from various known attacks. To the best of the author's knowledge and existing literature, no blockchain-based system uses PoA to combat the evils of COVID-19, such as manipulating critical information related to hospital amenities and patient data. CovidBChain minimizes storage and retrieval time, the number of messages exchanged, and transaction cost.

1.2 | Research contributions

A secure and efficient Ethereum-based framework Covid-19 big data blockchain (CovidBChain), to maintain the security, transparency, and integrity of Covid-19 data is proposed. Compared with state-of-the-art works, the contribution of the article is listed as follows:

1. It provides a blockchain-enabled framework CovidBChain to ensure transparency, integrity and security of healthcare data. It helps in preventing the government authorities and hospital administrators from manipulating the Covid-19 data due to political pressure or to earn a significant profit. We also eliminate typical cloud or centralized storage problems by storing healthcare data on a decentralized storage system known as IPFS.
2. CovidBChain services (such as authentication, segregation, storage, querying, and access control) ensure transparency of Covid-19 statistics and hospital amenities by putting the brakes on black-marketing amenities such as oxygen cylinders, ICU beds, ventilators, and PPE kits.
3. In CovidBChain, the proof-of-work (PoW) consensus mechanism is replaced with a proof-of-authority (PoA) that minimizes transaction cost, the number of messages exchanged, response time and improves scalability.

4. The formal security analysis of the CovidBChain is provided through the standard “BAN logic protocol.” Informally the security of CovidBChain is proved through mathematical equations and methods. The conducted security analyses confirm the security of CovidBChain against the different types of potential attacks (impersonation, replay, privileged insider, session key disclosure).
5. Finally, the result analysis shows that data upload and retrieval time, gas consumption, and network overhead is more when we store complete information on blockchain than on a combination of on-chain/off-chain (blockchain and IPFS) storage. Results show that high scalability can be achieved by compromising a little on security.

The article is organized as follows. The literature survey is summarized in Section 2. The preliminaries are discussed in Section 3. Section 4 discusses the detailed architecture of CovidBChain. It provides details about the various layers of the system, such as presentation, service, blockchain, and data. Various use case scenarios are discussed in Section 5. Section 6 discusses results and security analysis. It compares CovidBChain with different existing approaches. A use-case of the proposed model is discussed in Section 7. Section 8 concludes the article.

2 | LITERATURE SURVEY

Xu et al.¹⁵ introduce BeepTrace, a blockchain-based privacy-preserving contact tracing scheme to disassociate the user ID and location information. The proposed solution shows higher privacy and security with additional benefits of battery-friendly and worldwide acceptability. Kumar et al.¹⁶ proposed a model that collects data from various resources that trains a deep learning model using a blockchain-based federated model. The data is authenticated through blockchain, and deep learning trains the model while keeping the privacy intact of the organizations. In Reference 17, the authors proposed a blockchain-based framework so that users can securely check the uploaded data and tackle the information privacy issue by appending the digital signature.

In Reference 18, Marbough et al. explain blockchain's power to improve clinical trials by bringing down delays in approvals from regulatory authorities. They use smart contracts and oracles to track new infections, new deaths, and recovered cases obtained from various resources. Their solution is generic enough to counter malaria, HIV, and TB. Chang and Park¹⁹ discuss the importance of blockchain's role in a pandemic to fight against pandemics such as Covid-19 in terms of four aspects. First, the Covid-19 data are directly stored on the blockchain without needing an intermediary. Second, blockchain makes the various types of donations, such as medical equipment and monetary donation, transparent. Third, blockchain's immutable nature makes it very difficult for a malicious entity to manipulate and delete the information, which increases information reliability in society. Fourth, the blockchain stores lab reports and prescriptions of the patient online. Thus, it eliminates the risk of infection spread through reports or physical contact. Torky and Hassanien²⁰ propose a blockchain-based framework that examines the possibility of using blockchain for peer-to-peer, decentralized and timestamped storage. It can detect and verify the infected cases of Covid-19. It enables people to predict the risk of Covid-19 virus infection within individual conglomerates or public places via a new p2p-mobile application. The proposed framework consists of four components: Infection checker subsystem, blockchain platform, P2P-mobile application, and extensive area monitoring system.

In Reference 21, the authors explained how we could optimize the concept of “immunity certificates” through blockchain. They discuss the use of smart contracts to make the public key replaceable with an anonymous key. It also outlines how immunity certificates encourage forgery and incentivize people to come in contact with the person who already develops the immunity. The authors also show their concern about the chances of recursive infection, as there is no concrete evidence that shows the person cannot have Covid-19 infection more than once. The authors also raise their concern about social division due to immune and unimmune people. The authors proposed a blockchain-based data-sharing approach in Reference 22 that offers the blockchain's reliability, integrity, and decentralization properties. The proposed framework was designed based on blockchain technologies for organ/tissue transplantation by using the Hyperledger Fabric environment.

Mashamba-Thompson and Crayton²³ talk about the anomalies of current healthcare systems like overburdening, lack of resources, and fragile surveillance systems, making it harder to cope with the Covid-19 pandemics. It proposes a blockchain and artificial amalgamated approach used for tracking and self-testing of Covid-19 probabilities.

Kaur et al.²⁴ presented a blockchain-based framework using Hyperledger Fabric that provides secure storage, sharing, and querying electronic health records (EHRs). As a state database, they employed CouchDB (a document store database), where the search is based on both keys and data values rather than just keys. In Reference 25, the authors propose a generic scheme called VaCoChain. It combines blockchain, uncrewed aerial vehicles, and fifth-generation communication technology for the timely distribution of vaccines in Covid-19. The study in Reference 26 proposes a telecare medical information system based on blockchain for preserving medical data. It consists of a medical sensor area authentication protocol and a social network information transmission protocol. Table 1 presents blockchain-enabled Covid-19 mitigation solutions with pros and cons. The existing systems do not ensure the accuracy, transparency, and integrity of EHRs when the government authorities and hospital administrators manipulate the Covid-19 data due to political pressure and to earn a significant profit.

TABLE 1 Current blockchain-enabled Covid-19 mitigation solutions with pros and cons

Year	Title	Pros	Cons
2020	BeepTrace: Blockchain-enabled privacy-preserving contact tracing for COVID-19 pandemic and beyond ¹⁵	Higher privacy and security with additional benefits of battery-friendly	Location privacy cannot be well-guaranteed due to the use of Geo key
2021	Blockchain-federated learning and deep learning models for covid-19 detection using CT imaging ¹⁶	Improves recognition accuracy and preserves privacy	Suffers from massive communication costs and under-performs when data heterogeneity is high
2020	Blockchain-enable contact tracing for preserving user privacy during Covid-19 outbreak ¹⁷	Privacy is guaranteed by increasing the number of identifiers	High network traffic and the blockchain load
2020	Blockchain for COVID-19: Review, opportunities, and a trusted tracking system ¹⁸	Ensuring trustworthiness and immutability of Covid-19 data	Use of oracles makes the system vulnerable. They can supply manipulated data
2020	How can blockchain help people in the event of pandemics such as the COVID-19 ¹⁹	Provides immutability and transparency to the donation process	Implementation of the proposed system is missing
2020	COVID-19 blockchain framework: Innovative approach ²⁰	Predict the risk of virus infection within individual conglomerates or public places	Integration of the infection verifier subsystem and a mass surveillance system is unclear
2020	A novel method to ensure the security of the shared medical data using smart contracts: Organ transplantation sample ²¹	Ensure secure and effective EHR data sharing with transparency and accountability	Privacy issue of health data is not discussed
2020	Optimizing the implementation of COVID-19 "immunity certificates" using blockchain ²²	Immunity certificates discriminate between infected and uninfected	Encourage forgery and incentivize people to seek infection
2020	Blockchain and artificial intelligence technology for novel coronavirus disease-19 self-testing ²³	Low cost and restrict the transmission of infection	Deployment details of the proposed system are missing
2020	Blockchain-based framework for secured storage, sharing, and querying of electronic healthcare records ²⁴	It uses CouchDB, suitable for handling complex queries	Scaling capability of the model is not discussed
2021	VaCoChain: Blockchain-based 5G-assisted UAV vaccine distribution scheme for future pandemics ²⁵	Ensures timestamped documentation of vaccinated persons with chronology, auditability, and transparency of supply chain checkpoints	The proposed scheme not ensures perfect forward secrecy and privacy of communication among healthcare stakeholders
2021	Blockchain-based forward supply chain and waste management for Covid-19 medical equipment and supplies ²⁷	Proposed solution assist authorities in ensuring that the medical wastes are disposed of properly	Incentive mechanism for the hospitals to dispose of medical waste is missing
2022	Blockchain-based supply chain traceability for Covid-19 personal protective equipment ²⁸	Provides traceability of personal protective equipment for medical staff combating with COVID-19	Real-time testing of the proposed solution on a Ethereum network, still needs to done
2021	MedHypChain: A patient-centered interoperability Hyperledger-based medical healthcare system: Regulation in COVID-19 pandemic ²⁹	Privacy-preserving medical system provides anonymity, confidentiality, unforgeability, and traceability	Does not address the elucidation key escrow problem
2021	SAI-BA-IoMT: Secure AI-based blockchain-assisted Internet of medical things tool to moderate the outbreak of Covid-19 crisis ³⁰	Proposes a secure AI-based blockchain-assisted healthcare system to combat COVID-19 crisis	Implementation details of the proposed system is missing
2022	IoMT: A Covid-19 healthcare system driven by federated learning and blockchain ³¹	Provides trust and authenticity to public media communication and deliver an authentic method to disseminate COVID-19 information	Scalability of proposed consensus protocol is not discussed
2022	Privacy-preserving COVID-19 contact tracing solution based on blockchain ³²	Provides an efficient privacy-preserving automated COVID-19 tracing framework	Not consider any location data into the tracing framework and can only achieve passive security in the semi-honest mode

TABLE 2 Comparison of blockchain-enabled schemes with CovidBChain

Parameters	18	24	25	27	28	29	30	CovidBChain
Consensus	PoS	RAFT	QBFT	PoW	PoW	PBFT	Not mentioned	PoA
Blockchain	Ethereum	Hyperledger Fabric	Hyperledger Besu	Ethereum	Ethereum	Hyperledger Caliper	Not mentioned	Ethereum
Scalability	Average	High	High	Low	Low	High	Not mentioned	High
Energy consumption	Low	Low	Low	High	High	Low	Not mentioned	Low
Implementation	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes
Threat model	No	No	No	Yes	No	Yes	No	Yes
Decentralized external storage	No	Yes	Yes	Yes	Yes	No	No	Yes
Security analysis	Yes	No	Yes	Yes	Yes	Yes	No	Yes
Communication complexity	Low	Low	Low	High	High	Low	Not mentioned	Low

Ahmad et al.²⁷ propose a decentralized blockchain-based approach to automate the forward supply chain activities for Covid-19 medical equipment and make it possible for all parties involved in waste management to share information in a secure, transparent, traceable, and reliable manner. In Reference 28, a generic blockchain-based approach is presented for transforming PPE supply chain processes via smart contracts. Kumar and Chand²⁹ suggested a blockchain-based solution for protecting Covid-19 patients' privacy. The proposed technique takes advantage of identity-based broadcast group signcryption. In the healthcare system in the Covid-19 crisis, the authors in Reference 30 suggest a secure AI-based blockchain-assisted IoMT (SAI-BA-IoMT) approach. It also looks at the potential chaos that could follow the epidemic in the world. Samuel et al.³¹ provide trust and authenticity to public media communication and deliver an accurate method to disseminate Covid-19 information. The authors proposed a blockchain-based contact tracing framework.³² The zero-knowledge proof is used to protect the privacy of close contacts. It also uses an aggregate signature scheme to improve signature verification efficiency. Table 2 compares the strength and weaknesses of existing blockchain-based solutions with CovidBChain.

3 | PRELIMINARIES

This section discusses various preliminaries, such as the one-way hash function and elliptic curve cryptography used in the CovidBChain. After that, we discuss security and performance in the threat model.

3.1 | Cryptographic hash function

It is a function which takes arbitrary length input and produces a fixed-length output, $h : \{0, 1\}^* \rightarrow \{0, 1\}^l$ where h should be one way. Further given (x, y) cannot be found in polynomial time such that $h(x) = h(y)$ and computationally infeasible to find a pair $(x, y) : h(x) = h(y)$.

3.2 | Elliptic curve cryptosystem

The equation defines a nonsingular elliptic curve $(E) y^2 = x^3 + ax + b \pmod{p}$, where $a, b \in \mathbb{Z}_p^*$, $\mathbb{Z}^*$ is real numbers set, and p is a large prime such that $4a^3 + 27b^2 \neq 0$. The condition required for a nonsingular elliptic curve is $4a^3 + 27b^2 \neq 0$ and for singular elliptic curve condition is $4a^3 + 27b^2 = 0$.

3.3 | Threat model

The principles of well-known Dolev Yao³³ (DY) and Canetti and Krawczyk's^{34,35} (CK) threat model is followed in the design of CovidBChain. In the CovidBChain, data is sent across a public channel during the registration and authentication phase. Therefore, various security and performance considerations in the threat model^{36,37} are assumed to understand the capabilities of an adversary.

1. A malicious entity (M_E) knows the communication protocol completely, and s/he can compute different values if and only if M_E has all necessary credentials ($ID_i, ID_j, PWD_i, Net_{Adm}sk, R1_{NA}, R2_{NA}$).
2. Consider X and Y , two authorized users. In this case, if X can engage in criminal acts using Y 's credentials, then X is an opponent for Y in two ways (legal user and attacker).
3. To communicate with hospitals (HP_j), a genuine patient (PT_i) can act as a malicious entity (M_E) and send repeated requests using different illicit identities. An attacker aims to keep HP_j busy so that on-demand resources (doctors, amenities, and lab personnel) are not available to other authorized users.
4. The session key (SK_{ij}) is used to communicate between patients (PT_i) and the hospitals (HP_j). This key (SK_{ij}) is temporary and can only be determined successfully if both the sender and receiver agree on shared values and have all of the necessary criteria.
5. We consider an equation, $X = Y \oplus Z$. Here, M_E can obtain X if and only if M_E knows Z and Y , as shown in Algorithm 3. However, M_E cannot get X or Y or Z if s/he has only one value (Y or Z or X).
6. M_E can steal the smart card (SC_i). Then, using a sophisticated power analysis assault,³⁸ M_E attempts to retrieve the secret information contained in their memory.
7. M_E can guess one credential at a time. It means that M_E can proceed in the computation with only one parameter (e.g., user password, random number, or relevant credential) as a guessable value. Next, guessing more than one parameter/random number correctly in polynomial time is impossible.
8. All cryptographic primitives (symmetric/asymmetric, one-way hash, and elliptic curve cryptography) are secure, and M_E cannot break these algorithms.

4 | ARCHITECTURE OF COVIDBCHAIN

This section introduces the architecture of the CovidBChain and discusses its services. Figure 1 represents the layered architecture of CovidBChain, which has been consists of the user interface and registration module. The registration is carried out between users and the network administrator over a secure infrastructure. This module's primary purpose is to provide legitimacy to the users who want to interact with the framework and prevent the malicious actors from doing unwanted activities.

The service layer comprises authentication, access control, segregation, storage, and querying module. The access of authentic users is limited with a fine-grained access-control module. Segregation breaks the healthcare information into critical and noncritical data. Storage and querying are done through the blockchain and data layer in a distributed way. Table 3 lists the symbols used in this article.

4.1 | Presentation layer

The presentation layer consists of a user interface and registration module. Patients, doctors, lab staff, and hospitals can interact with the system through the user interface. The registration module consists of the patient and the hospital registration. Stakeholders need to register themselves through the registration module to work in orchestration with other entities.

4.1.1 | Registration module

In this module, a user registers with the Net_{Adm} to become a legal user to get services from different hospitals legitimately. Besides, a hospital should also perform the registration procedure with the Net_{Adm} , and after that, that hospital is permitted to provide services to legitimate users.

1. Patient registration

If a patient (PT_i) wants to receive a medical diagnosis. In that case, the PT_i must register their information with the network administrator (Net_{Adm}) and generate a private and public key to receive any diagnosis report from the hospital (HP_j). Algorithm 1 provides registration facility for the patient to access the system's services securely. It is shown with the help of steps 1–3.

Step 1: The PT_i sends a registration request to the Net_{Adm} . Initially, ID_i and password PWD_i are entered by the PT_i . It also generates a random number p_i and computes a hash value by concatenating p_i with ID_i , and sends the computed hash value (PT_{ID_i}) to the Net_{Adm} .

Step 2: A random number $R1_{NA}$ is chosen by the Net_{Adm} and computes $z_i = h(PT_{ID_i} || R1_{NA})$ using the PT_{ID_i} , which is received from the PT_i . The Net_{Adm} stores z_i into the smartcard (SC_i) and provides it to the PT_i in the blockchain, and the Net_{Adm} is put down PT_{ID_i} in a secure database (DB).

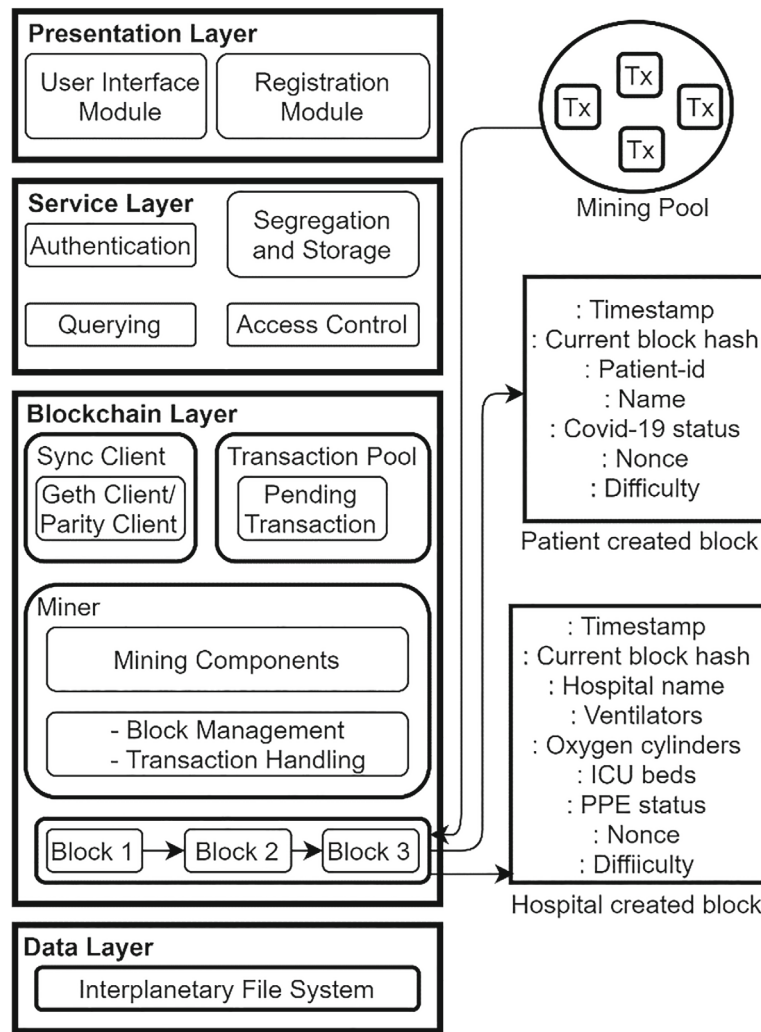


FIGURE 1 Layered architecture of CovidBChain

Step 3: The PT_i generates a random number PT_{ski} as the secret key after receiving the SC_i from Net_{Adm} . First PT_i computes $PTPWD_i = h(ID_i || PWD_i)$, $W_i = PTPWD_i \oplus p_i$, $X_i = h(ID_i || PWD_i || p_i) \oplus PT_{ski}$, $Y_i = h(p_i || PT_{ski}) \oplus z_i$ and $Z_i = h(p_i || PT_{ski} || z_i)$ and then, a public key $PT_{pki} = PT_{ski} \cdot G$ is generated by the PT_i and replaces $\{z_i\}$ with $\{Y_i\}$ in the SC_i . Finally, PT_i stores $\{W_i, X_i, Y_i, Z_i\}$ in the SC_i .

2. Hospital registration

A hospital (HP_j) must register with the Net_{Adm} to have a public and private key agreement with patients and exchange medical reports/prescriptions with other hospitals. The masked identity of the HP_j is shared amongst other system entities. Algorithm 2 illustrates the hospital registration process, and the detailed steps are as follows:

Step 1: A HP_j selects a distinctive identity ID_j and choose a random number HP_{skj} as its private key. Then, the HP_j calculates a masked identity $HP_{IDj} = h(ID_j || HP_{skj})$ and creates a public key $HP_{pkj} = HP_{skj} \cdot G$. HP_j sends HP_{IDj} to the Net_{Adm} .

Step 2: Once the Net_{Adm} receives the request message, the Net_{Adm} generates a random number ($R2_{NA}$) and access $\{PT_{IDi}\}$ from the database. After that, the Net_{Adm} calculates $Cert_j = h(HP_{IDj} || R2_{NA}) + Net_{Adm}sk \cdot HP_{pkj}$. The Net_{Adm} stores $Cert_j$ with HP_{IDj} and sends $\{Cert_j, PT_{IDi}\}$ to HP_j .

Step 3: Once the HP_j receives the message, the HP_j stores $\{Cert_j, PT_{IDi}\}$ in a secure database.

4.1.2 | User interface

In the healthcare domain, user experience is as crucial as in other domains because the well-being of patients is at stake. The importance of a well-designed user interface in hiding software complexity from end users cannot be overstated. The use case has been elaborated in Section 4. It highlights major features and capabilities while clarifying the software's clinical purpose for the average user (patient, doctor, nurse, lab staff).

TABLE 3 List of different notations

	Notation	Meaning
1.	PT_i	i th patient
2.	HP_j	j th hospital
3.	Net_{Adm}	Network administrator
4.	ID_i, ID_j	Identity of PT_i and HP_j
5.	PWD_i	Password of PT_i
6.	PT_{ski}, HP_{ski}	Secret keys of PT_i and HP_j
7.	$Net_{Adm}sk$	Secret key of Net_{Adm}
8.	z_i	Stores hash value of PT_{ID_i} and $R1_{NA}$
9.	$PTID_i$	Masked patient id
10.	$HPID_j$	Masked hospital id
11.	PT_{pki}	i th patient public key
12.	HP_{pkj}	j th hospital public key
13.	T_1, T_2	Timestamps
14.	SC_i	Smartcard
15.	$R1_{NA}, R2_{NA}$	Random numbers generated by Net_{Adm}
16.	R_{HP}	Random number generated by hospital
17.	PT_{pki}, HP_{pki}	Public keys of PT_i and HP_j
18.	$Cert_i, Cert_j$	Certificates of PT_i and HP_j
19.	G	A base point for elliptic curve
20.	$PTID_i, HPID_j$	Pseudo-identities of PT_i and HP_j
21.	$h(*)$	One-way hash function
22.	$\otimes$	XOR operation
23.	$\parallel$	Concatenation operation

4.2 | Services layer

This layer provides services such as authentication, access control, segregation and storage, and querying.

4.2.1 | Authentication

Algorithm 3 presents authentication for CovidBChain. If a registered user wants to access the functionalities of the healthcare system, the authentication layer will authenticate whether the user is legitimate or not. If a PT_i seeks to undergo a health diagnosis process, the PT_i and HP_j should set up a session key. The authentication process of CovidBChain is illustrated in Algorithm 3, and the detailed steps are as follows:

Step 1: The PT_i inputs his/her ID_i , PWD_i , and SC_i . Then, the SC_i computes $PTPWD_i = h(ID_i \parallel PWD_i)$, $p_i = PTPWD_i \oplus W_i$, $PTID_i = h(p_i \parallel ID_i)$, $PT_{ski} = h(ID_i \parallel PWD_i \parallel p_i) \oplus X_i$, $Z_i = h(p_i \parallel PT_{ski}) \oplus Y_i$, and $Z_i^* = h(p_i \parallel PT_{ski} \parallel Z_i)$. Then, the SC_i checks whether $Z_i^* = Z_i$. If it matches, the PT_i produce a timestamp T_1 and encrypts the messages $M_1 = \{(Z_i \parallel PTID_i \parallel T_1) + PT_{ski} \cdot HP_{pkj}\}$ and computes $M_{ap} = h(Z_i \parallel PTID_i)$. Next, the PT_i sends a message $\langle M_1, M_{ap}, T_1 \rangle$ to HP_j using a public channel.

Step 2: When the message $\langle M_1, M_{ap}, T_1 \rangle$ is received by hospital (HP_j), the HP_j decrypts $(Z_i \parallel PTID_i \parallel T_1) = M_1 - HP_{skj} \cdot PT_{pki}$. Then, the HP_j retrieves $PT_{ID_i}^*$ from the database and compares $PTID_i = PT_{ID_i}^*$. If the value matches, the HP_j calculates $M_{ap}^* = h(Z_i \parallel PTID_i)$ and checks whether $M_{ap}^* = M_{ap}$. If they are equal, the HP_j creates a random number R_{HP} and T_2 timestamp to calculate $E_i = R_{HP} \oplus X_i$, $M_{amc} = h(HPID_j \parallel R_{HP} \parallel T_2)$. Then, the HP_j creates a session key $SK_{ij} = h(PTID_i \parallel HP_{MID_j} \parallel Z_i \parallel R_{HP})$. Finally, HP_j sends message $\langle E_i, M_{amc}, SK_{ij} \rangle$ to PT_i through an open channel.

Step 3: When PT_i receives the message from the HP_j . The PT_i calculates $R_{HP} = E_i \oplus X_i$, and $M_{amc}^* = h(HPID_j \parallel R_{HP} \parallel T_2)$. Then, the PT_i checks whether $M_{amc}^* = M_{amc}$. If they are equal, the PT_i calculates a session key $SK_{ij} = h(PTID_i \parallel HPID_j \parallel Z_i \parallel R_{HP})$.

Algorithm 1. Secure patient registration

Input: $ID_i, PWD_i, p_i, R1_{NA}$
Output: Patient registration successful

- 1: **procedure** *Patient_Registration*(PT_i) ▷ Patient masked id is calculated and sent to the network administrator
- 2: $PT_{ID_i} = h(p_i \parallel ID_i);$
- 3: $Net_{Adm} \leftarrow PT_{ID_i}$ ▷ Hash is calculated by network administrator using masked id
- 4: $z_i = h(PT_{ID_i} \parallel R1_{NA})$ ▷ Network administrator stores the value of z_i and patient masked id
- 5: $SC_i = z_i;$
- 6: $DB = store(PT_{ID_i})$ ▷ Patient calculates $PTPWD_i, W_i, X_i, Y_i, Z_i$
- 7: $PTPWD_i = h(ID_i \parallel PWD_i);$
- 8: $W_i = PTPWD_i \oplus p_i;$
- 9: $X_i = h(ID_i \parallel PWD_i \parallel p_i) \oplus PT_{ski};$
- 10: $Y_i = h(p_i \parallel PT_{ski}) \oplus z_i;$
- 11: $Z_i = h(p_i \parallel PT_{ski} \parallel z_i)$ ▷ Patient generates public key and replace z_i with Y_i
- 12: $PT_{pki} = PT_{ski}.G;$
- 13: $Y_i = Z_i$ ▷ Patient stores W_i, X_i, Y_i, Z_i in smartcard
- 14: $SC_i = W_i, X_i, Y_i, Z_i;$
- 15: **end procedure**

Algorithm 2. Secure hospital registration

Input: ID_j, HP_{skj}
Output: $Cert_j$

- 1: **procedure** *Hospital_Registration*(HP_j) ▷ Hospital masked id and public key is calculated and sent to the network administrator
- 2: $HP_{ID_j} = h(ID_j \parallel HP_{skj})$
- 3: $HP_{pkj} = HP_{skj}.G$
- 4: $Net_{Adm} \leftarrow HP_{ID_j}$ ▷ Network administrator calculates certificate for the hospital
- 5: $Cert_j = h(HP_{ID_j} \parallel R2_{NA}) + Net_{Adm}.sk. HP_{pkj}$ ▷ Network administrator stores certificate and masked identity
- 6: $Net_{Adm} = store(Cert_j, HP_{ID_j})$ ▷ Network administrator sends certificate masked identity to the hospital
- 7: $HP_j \leftarrow (Cert_j, PT_{ID_i})$
- 8: **end procedure**

4.2.2 | Access control

The access control module provides fine-grained control over healthcare data. So, a requester will access only that part of the data which the owner permits. It is neither feasible nor efficient to store complete health care data onto the blockchain. Thus, data needs to be segregated.

4.2.3 | Segregation and storage

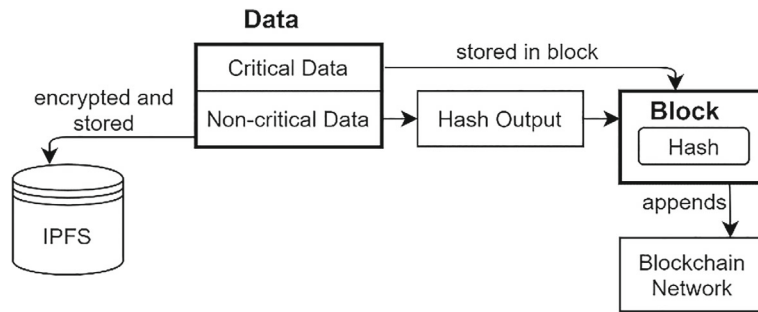
To handle the enormous volume of healthcare data and to preserve the privacy of the patient and the hospital, their complete information cannot be stored on the blockchain. Thus, it is first segregated and then stored (Figure 2). So, data are divided into two parts "critical" and "noncritical." For example, critical data for a patient in the healthcare domain is his "name," "identity number," and timestamp. For the hospital, it is the hospital name, number of available "ventilators," "oxygen cylinders," "ICU beds," "PPE status," "state." Noncritical fields include "date of birth," "weight," "Covid-19 status," "prescription," and "reports" of the patient. So, the critical fields that are threatened to be manipulated will be bundled into a block and stored as a field of transaction in the blockchain. In contrast, the noncritical data after encryption is stored off-chain, that is, on the IPFS. The hash value of noncritical data is stored as a part of a block with critical data. Thus, critical data becomes tamper-proof and traceable because of the inherent property of the blockchain. The segregation and storage module comprises of three algorithms.

1. Segregating the data into critical and noncritical parts.
2. Protecting the data using symmetric encryption.
3. Storing the critical data in the blockchain and noncritical data on IPFS.

Algorithm 3. Authentication in CovidBChain

Input: ID_i, PWD_i, SC_i
Output: SK_{ij}

- 1: **procedure** CovidBChain_Authentication() ▷ Smart card computes
- 2: $PTPWD_i = h(ID_i \parallel PWD_i)$
- 3: $p_i = PTPWD_i \oplus W_i$
- 4: $PT_{ID_i} = h(p_i \parallel ID_i)$
- 5: $PT_{ski} = h(ID_i \parallel PWD_i \parallel p_i) \oplus X_i$
- 6: $Z_i = h(p_i \parallel PT_{ski}) \oplus Y_i$
- 7: $Z_i^* = h(p_i \parallel PT_{ski} \parallel z_i)$ ▷ Smart card checks if Z_i^* and Z_i are equal
- 8: **if** ($Z_i^* == Z_i$)
- 9: $M_1 = (z_i \parallel PT_{ID_i} \parallel T_1) + PT_{ski}.HP_{pkj}$
- 10: $M_{ap} = h(z_i \parallel PT_{ID_i})$ ▷ Patient sends a message to hospital
- 11: $HP_j \leftarrow PT_i.send(M_1, M_{ap}, T_1)$ ▷ Hospital checks if $PT_{ID_i}^*$ and PT_{ID_i} are equal
- 12: **if** ($(PT_{ID_i}^* == PT_{ID_i}) \& \& (M_{ap}^* == M_{ap})$)
- 13: $E_i = R_{HP} \oplus X_i$
- 14: $M_{amc} = h(HP_{ID_j} \parallel R_{HP} \parallel T_2)$ ▷ hospital generates a session key
- 15: $SK_{ij} = h(PT_{ID_i} \parallel HP_{MID_j} \parallel z_i \parallel R_{HP})$ ▷ hospital sends a message to the patient
- 16: $PT_i \leftarrow HP_j.send(E_i, M_{amc}, SK_{ij})$ ▷ Patient computes R_{HP} and M_{amc}^*
- 17: $R_{HP} = E_i \oplus z_i$
- 18: $M_{amc}^* = h(HP_{ID_j} \parallel R_{HP} \parallel T_2)$
- 19: **if** ($M_{amc}^* == M_{amc}$)
- 20: $SK_{ij} = h(PT_{ID_i} \parallel HP_{ID_j} \parallel z_i \parallel R_{HP})$
- 21: **end procedure**

**FIGURE 2** Segregation and storage in CovidBChain

Algorithm 4 shows the pseudocode for the segregation of data. The data are stored in the specific file according to its nature, that is, critical and noncritical. The hash of the noncritical data is calculated using the SHA-256 algorithm, and stored in the blockchain for verification. Finally, the elliptic curve cryptography (ECC) algorithm generates a pair of asymmetric keys that will be used in Algorithms 5 and 7.

Algorithm 5 shows the pseudo-code for data protection and encryption of critical and noncritical data. The CovidBChain uses AES symmetric key algorithm to generate symmetric key Sym_{key} to encrypt the data. Critical data are stored in Crdata, and noncritical data are stored in Ncrdata. Next, the asymmetric public key Pub_K generated in Algorithm 4 is used to encrypt Sym_{key} .

Algorithm 6 shows the pseudocode for storing critical and noncritical encrypted data. The encrypted critical data are added to the block, and then that particular block is added to the blockchain using `append.blockchain()`. Similarly, encrypted noncritical data are stored in IPFS, and the SHA-256 hash value of noncritical data is stored in the blockchain. However, maintaining integrity and security during blockchain integration with IPFS is still a concern. This concern is discussed in the next section.

Algorithm 4. Data segregation**Input:** Data to be preserved, Description of data

```

1: procedure Segregate_Data()
2:   if(isData == critical)
3:     data is saved to critical file
4:   else
5:     Data is saved to noncritical file
6:     hashNcr = calculateSHA – 256(ofnoncriticalfile)
7:   endif
8:   Generate a pair of asymmetric keys (PubK, PrivK) using ECC algorithm
9: end procedure

```

▷ data is divided into critical and noncritical parts

▷ apply hashing over noncritical data

Algorithm 5. Data protection using encryption**Input:** Critical and noncritical data**Output:** Encrypted data and symmetric key

```

1: procedure Protect_Data()
2:   Symkey = AES()
3:   Symkey is used to encrypt the Data
4:   Data = Crdata || Ncrdata
5:   Crdata = Enc(Data, Symkey)
6:   Ncrdata = Enc(Data, Symkey)
7:   Pubk = Get the public key from asymmetric key pair generated in algorithm 1
8:   Generate a pair of asymmetric keys (PubK, PrivK) using ECC algorithm
9:   ESymkey = Enc(Symkey, pubk)
10: end procedure

```

▷ symmetric key is generated using AES encryption

Algorithm 6. Data storage**Input:** Encrypted critical and noncritical data**Output:** Blockchain transaction hash

```

1: procedure Store_Data()
2:   if(Encrypteddata == Crdata) then
3:     Crdata is added to the block bi
4:     append.blockchain(bi)
5:   else
6:     store.IPFS(Ncrdata)
7:     H_Ncrdata = append.blockchain(SHA – 256(Ncrdata))
8:   end procedure

```

▷ hash of encrypted noncritical data is stored in the blockchain for verification

4.2.4 | Querying

Figure 3 shows querying the noncritical data in CovidBChain while preserving its integrity. The information that the user wants to access is searched by querying the blockchain and the IPFS storage. First, the data queried from the IPFS storage are extracted and decrypted. Then it is hashed to obtain the hash value. While on the other side (i.e., blockchain) particular transaction is extracted from the blockchain block containing critical data corresponding to the query. Then hash of the record is taken from the transaction and compared with the hash value generated by data stored on IPFS. If they are equal, it proves that data integrity is maintained.

Algorithm 7 shows the pseudo-code for querying the stored data that extracts and decrypts the data. The transaction requested is extracted from the blockchain. Similarly, data extracted from IPFS corresponds to the transaction hash (T_x). Next, the generated asymmetric private key is

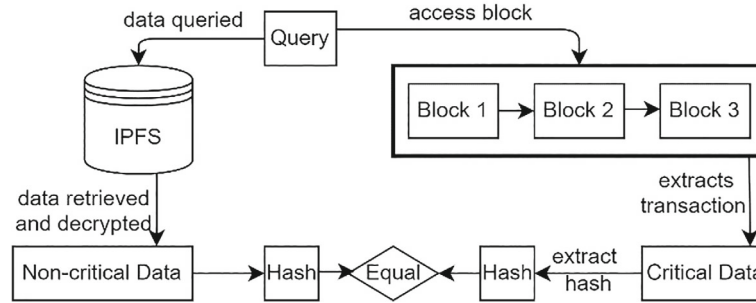


FIGURE 3 Querying the data (noncritical) in CovidBChain

Algorithm 7. Querying data

Input: Transaction Hash T_x

Output: Required Data

```

1: procedure Query_Data()                                ▷ access block from blockchain according to query
2:   access.blockchain(query)                             ▷ extract transaction from the block
3:   Extract.block( $T_x$ )
4:   Extract.IPFS( $T_x$ )
5:    $Priv_k$  = Get the private key from asymmetric key pair generated in algorithm 4    ▷ private key is used to decrypt  $ESym_{key}$ 
6:    $Sym_{key} = Decr(ESym_{key}, Priv_k)$ 
7:   HashS = stored hash on blockchain
8:   HashE = extracted hash from IPFS
9:   if (HashS = HashE) then
10:    return D
11:  else
12:    return null
13:  end if
14: end procedure

```

used to decrypt the symmetric key. The symmetric key decrypts the data stored in IPFS. The SHA-256() is applied over the data stored in IPFS, that is, HashE. If two hashes match, then the data are sent to the requestor.

4.3 | Blockchain layer

Ethereum³⁹ blockchain is used to implement the CovidBChain framework. Ethereum is a blockchain-based open-source platform that allows developers to create and deploy distributed applications. It allows developers to create any application they want on a single decentralized platform. Ethereum focuses on running code for any distributed application deployed on its network. Various components of the Ethereum ecosystem are discussed as follows:

Block: A block is the basic data-containing unit of a blockchain. The previous block's hash value serves as a link between it and its prior block. It contains critical data fields like previous hash, current hash, time, data: patient-id, ventilators, oxygen cylinder, ICU beds, and so forth.

Sync client: A client is an entity/program that facilitates communication with the Ethereum network. A network consists of nodes, and these nodes of the network are also known as clients. Parity and Go-Ethereum can act as an Ethereum node.

Miner: It is a special node in the blockchain network equipped with very high computation power. Its job is to add the block to the blockchain. In the Ethereum network, miners solve a computer-intensive puzzle. Once the majority of nodes show their consent to the solved puzzle, the miner is eligible to add the block that consists of transactions to the blockchain network.

Mining pool: Each miner in the network maintains the mining pool. It consists of unconfirmed transactions; miners pick these unconfirmed transactions from the mining pool and add them to the block. Once the block gets mined, all the blockchain transactions become permanent and added to the blockchain.

Smart contract: Smart contracts play an essential role in the blockchain-enabled system. It facilitates the implementation of policies for the transfer of assets in a decentralized network. They are immutable, distributed, and eliminate the need for any intermediary. Various use cases implemented through smart contracts are discussed in Section 4.

4.4 | Data layer

To handle an enormous volume of data, CovidBChain segregates and stores the data. Current research uses IPFS to store noncritical data. IPFS is a peer-to-peer protocol that is created to make the web faster, safer, and more open. It works like how bit-torrent works. In the traditional centralized web, information is accessed using location-based addressing, while IPFS uses content-based addressing. Security and de-duplication of files make IPFS very efficient. In IPFS, files are stored as IPFS objects, and each IPFS object can store up to 256 kB of data; it also contains links to other IPFS objects. In IPFS, a large file greater than 256 kB has to be split up into multiple IPFS objects, and afterward, the system creates an empty IPFS object that contains a link to the splitted files. IPFS ensures that the file and its entire history are accessible to other nodes in the network.

5 | COVIDBCHAIN STAKEHOLDERS AND THEIR USE CASE SCENARIOS

Interaction between different stakeholders of the health care system, such as patients, hospitals, doctors, lab staff, and so forth, is discussed in this section. Any interaction between stakeholders of the system is recorded in the blockchain through smart contracts. The current research proposes locking and unlocking scripts to elaborate use case scenarios. A locking script is a constraint on crucial information, ensuring that no one can access the information without permission. An unlocking script is used to satisfy the restriction placed on vital information by a locking script. So that information can be accessed securely and at the fine-granularity level. Section 5.1 elaborates on the process of reserving an appointment with the medical practitioner. Section 5.2 presents scripts for the scenario when a provider requests the patient test reports conducted earlier as part of another practitioner's treatment. Section 5.3 discusses the process of dispense of medication information to the patient and the pharmacist.

5.1 | Patient visits to the doctor

To reserve an appointment with a medical practitioner, the patient requests an appointment addressing the medical practitioner's office.

$Provider_{Adr} : (Patient_{Adr}, \text{Sign}((\text{Request}), Patient_{PriK}))$

The practitioner's office receives the patient's message and extracts his address $Patient_{Adr}$. After receiving the patient's request, the practitioner's office sends a response message to the patient. When a patient receives the encrypted response message, he decrypts it using his private key to get the original response and the provider's public key. The pseudo-code for this scenario is written in script 1.

Patient: $(\text{Sign}(\text{Hash}(Provider_{PubK}), Patient_{PubK}), (\text{Sign}(\text{Hash}(\text{Response}), Patient_{PubK}))$

Script 1. Reserve appointment with the doctor

Require:

$Provider_{Adr} : (Patient_{Adr}, \text{Sign}((\text{Request}), Patient_{PriK}))$

Provider extracts: $Patient_{Adr}$ && Request

Locking Script:

Patient: $(\text{Sign}(Provider_{PubK}), Patient_{PubK}), (\text{Sign}(\text{Response}), Patient_{PubK})$ && <time>

Unlocking Script:

$Provider_{PubK} : Patient_{PriK}(\text{Sign}(Provider_{PubK}), Patient_{PubK})$ && $Patient_{PriK}(\text{Sign}(\text{Response}), Patient_{PubK})$ && <time>

main():

if time remains && status = 1 then

Response: "Accept request";

else

destruct contract;

Suppose a doctor receives requests from more than one patient. In this case, a smart contract is initiated for allocating appointments to more than one requestor. This contract is triggered by all the patients and executed by the provider as described in script 2.

Script 2. Doctor receives request from more than one patient

Require:

Provider_{Adr}: (Patient-1_{Adr}, Sign((Request), Patient_{PriK}))

Provider extracts: Patient-1_{Adr} && Request

Provider_{Adr}: (Patient-2_{Adr}, Sign((Request), Patient_{PriK}))

Provider extracts: Patient-2_{Adr} && Request

Provider_{Adr}: (Patient-3_{Adr}, Sign((Request), Patient_{PriK}))

Provider extracts: Patient-3_{Adr} && Request

Locking Script:

Patient-1: (Sign(Provider_{PubK}), Patient-1_{PubK}), (Sign(Response), Patient-1_{PubK}) && <time>

Patient-2: (Sign(Provider_{PubK}), Patient-2_{PubK}), (Sign(Response), Patient-2_{PubK}) && <time>

Patient-3: (Sign(Provider_{PubK}), Patient-3_{PubK}), (Sign(Response), Patient-3_{PubK}) && <time>

Unlocking Script:

Provider_{PubK} && Response:

Patient-1_{PriK}(Sign(Provider_{PubK}), Patient-1_{PubK}) && Patient-1_{PriK}(Sign(Response), Patient-1_{PubK}) && <time>

Provider_{PubK} && Response:

Patient-2_{PriK}(Sign(Provider_{PubK}), Patient-2_{PubK}) && Patient-2_{PriK}(Sign(Response), Patient-2_{PubK}) && <time>

Provider_{PubK} && Response:

Patient-3_{PriK}(Sign(Provider_{PubK}), Patient-3_{PubK}) && Patient-3_{PriK}(Sign(Response), Patient-3_{PubK}) && <time>

main():

if time remains && status = 1 then

Response: "Patient-1 confirms appointment";

Response: "Patient-2 confirms appointment";

Response: "Patient-3 confirms appointment";

else

deconstruct contract;

5.2 | Provider is requesting further tests from the patient

Suppose a new practitioner wants to access the prescriptions of another practitioner. In that case, he sends a request to the patient for asking the reports of the test conducted by the other practitioner.

Patient_{Adr}: (Provider_{Adr}, Sign((NewDocRequest), Provider_{PriK}))

When a patient receives the request, he extracts *Provider_{Adr}* from the received message and sends his response and *Patient_{PubK}* to the provider.

Provider: (Sign(Patient_{PubK}), Provider_{PubK}), (Sign(Response), Provider_{PubK}))

The provider extracts *Patient_{PubK}* and responds by decrypting the received message by his *Provider_{PriK}*. This is explained in script 3.

Script 3. New practitioner wants to access the prescriptions of another practitioner

Require:

Patient_{Adr}: (Provider_{Adr}, Sign(NewDocRequest), Provider_{PriK})

Patient extracts: Provider_{Adr} && NewDocRequest

Locking Script:

Provider: (Sign(Patient_{PubK}), Provider_{PubK}), (Sign(Response), Provider_{PubK}) && <time>

Unlocking Script:

Patient_{PubK} && Response:

Provider_{PriK}(Sign(Provider_{PubK}), Patient_{PubK}) && Provider_{PriK}(Sign(Response), Patient_{PubK}) && <time>

main():

```

if time remains && status = 1 then
Response: "Document reached";
else
destruct contract;

```

The hospital initiates smart contracts for their patients who want to undergo some medical procedure. The smart contract sets the appointment time for each patient, and the instructions addressed to them. The patients send a request to the providers to get some medical procedures. Provider extracts the patient address and request by decrypting it with the help of the patient private key.

Provider_{Adr}: (*Patient* – 1_{Adr}, Sign((Request), *Patient_{PriK}*))

Provider extracts: *Patient* – 1_{Adr} && Request

The provider sent his response and public key to the requesting patients through a locking script.

Patient-1: (Sign(*Provider_{PubK}*), *Patient* – 1_{PubK}), (Sign(Response), *Patient* – 1_{PubK})

The patient decrypts the provider's public key and responds to his request through his private key, as shown in script 4.

Script 4. Patient wants to undergo medical procedure

Require:

Provider_{Adr}: (*Patient*-1_{Adr}, Sign(Request), *Patient_{PriK}*)

Provider extracts: *Patient*-1_{Adr} && Request

Provider_{Adr}: (*Patient*-2_{Adr}, Sign(Request), *Patient_{PriK}*)

Provider extracts: *Patient*-2_{Adr} && Request

Provider_{Adr}: (*Patient*-3_{Adr}, Sign(Request), *Patient_{PriK}*)

Provider extracts: *Patient*-3_{Adr} && Request

Locking Script:

Patient-1: (Sign(*Provider_{PubK}*), *Patient*-1_{PubK}), (Sign(Response), *Patient*-1_{PubK}) && <time>

Patient-2: (Sign(*Provider_{PubK}*), *Patient*-2_{PubK}), (Sign(Response), *Patient*-2_{PubK}) && <time>

Patient-3: (Sign(*Provider_{PubK}*), *Patient*-3_{PubK}), (Sign(Response), *Patient*-3_{PubK}) && <time>

Unlocking Script:

Provider_{PubK} && Response: *Patient*-1_{PriK}(Sign(*Provider_{PubK}*), *Patient*-1_{PubK}) && *Patient*-1_{PriK}(Sign(Response), *Patient*-1_{PubK}) && <time>

Provider_{PubK} && Response: *Patient*-2_{PriK}(Sign(*Provider_{PubK}*), *Patient*-2_{PubK}) && *Patient*-2_{PriK}(Sign(Response), *Patient*-2_{PubK}) && <time>

Provider_{PubK} && Response: *Patient*-3_{PriK}(Sign(*Provider_{PubK}*), *Patient*-3_{PubK}) && *Patient*-3_{PriK}(Sign(Response), *Patient*-3_{PubK}) && <time>

main():

```

if time remains && status = 1 then

```

```

Response: "Patient-1 confirms X-Ray appointment";

```

```

Response: "Patient-2 confirms MRI appointment";

```

```

Response: "Patient-3 confirms City Scan appointment";

```

```

else

```

```

destruct contract;

```

5.3 | Doctors prescribes medication

Dispense of medication by the practitioner to the patient and pharmacist. The doctor initiates a smart contract addressing the patient and pharmacist. So that they can access the prescription.

Patient && Pharmacist: (Sign(Prescription), *Patient_{PubK}*), (Sign(Prescription), *Pharmacist_{PubK}*).

The pharmacist and patient can decrypt the prescription using their private keys to dispense and collect the medicines.

Patient_{PriK}(Sign(Prescription), *Patient_{PubK}*) && *Pharmacist_{PriK}*(Sign(Prescription), *Pharmacist_{PubK}*).

Pseudo code for this scenario is written in script 5.

Script 5. Dispense of medication to the patient and pharmacist

Require:

Locking Script:

Patient & Pharmacist: $(\text{Sign}(\text{Prescription}), \text{Patient}_{\text{PubK}}), (\text{Sign}(\text{Prescription}), \text{Pharmacist}_{\text{PubK}}) \&\& \langle \text{time} \rangle$

Unlocking Script:

Prescription: $\text{Patient}_{\text{PriK}}(\text{Sign}(\text{Prescription}), \text{Patient}_{\text{PubK}}) \&\& \text{Pharmacist}_{\text{PriK}}(\text{Sign}(\text{Prescription}), \text{Pharmacist}_{\text{PubK}}) \&\& \langle \text{time} \rangle$

main():

if signature matches && status = 1 then

Response: "Medicine sent";

else

destruct contract;

The medication information is signed by the patient and his partner's public key before storing it on the blockchain.

$(\text{Sign}(\text{Hash}(\text{Medication information}), \text{Patient}_{\text{PubK}})), (\text{Sign}(\text{Hash}(\text{Medication information}), \text{Partner}_{\text{PubK}}))$.

The patient and his partner can only access the stored information through their private keys, as discussed in script 6.

Script 6. Registering the record of dispensing medication onto the blockchain

Require:

Locking Script:

Patient & Partner: $(\text{Sign}(\text{Medication information}), \text{Patient}_{\text{PubK}}), (\text{Sign}(\text{Medication information}), \text{Partner}_{\text{PubK}}) \&\& \langle \text{time} \rangle$

Unlocking Script:

Prescription: $\text{Patient}_{\text{PriK}}(\text{Sign}(\text{Medication information}), \text{Patient}_{\text{PubK}}) \&\& \text{Partner}_{\text{PriK}}(\text{Sign}(\text{Medication information}), \text{Partner}_{\text{PubK}}) \&\& \langle \text{time} \rangle$

main():

if signature matches && status = 1 then

Response: "Medicine information recorded";

Response: "Procedure stored";

else

destruct contract;

6 | RESULTS AND SECURITY ANALYSIS

This section discusses the results of implementing CovidBChain on the Ethereum official test network Ropsten⁴⁰ for PoW consensus and Goerli⁴¹ for PoA consensus on various parameters such as upload and retrieval time, number of messages exchanged, and transaction cost parameters. We also perform the security analysis for authentication, registration and session key agreement of CovidBChain against various attacks.

6.1 | Result analysis

To show that the proposed framework can handle the colossal size data, that is, (big data), which consists of structured, semi-structured and unstructured data. We simulate our work on structured and unstructured data. Dataset⁴² contains covid-19 data of 214 countries which consists of 136,688 rows and 67 columns. We extracted the data of the country India, which consists of approximately 44,800 entries. Dataset⁴³ consists of 7470 chest X-ray multimedia images.

As discussed in Section 4.2.3, the Covid-19 data for patients and hospitals is divided into two categories, that is, critical and noncritical. Critical data fields for patients are patient-id, name, timestamp, and for the hospital, it is hospital name, state name, ventilator status, oxygen cylinder, ICU beds, and so forth. In contrast, noncritical data fields for patients are date of birth, weight, covid-19 status, prescription, and reports, and for the hospital, it is total_cases, new_cases, total_deaths, new_deaths, weekly_icu_admission, median_age, smoke_status, and so forth. Critical data's integrity, transparency, and pseudo-anonymity are maintained by storing it on the blockchain.

Figures 4–9 illustrate that if complete data (critical and noncritical) is stored on the blockchain, it is expensive in terms of space, time and cost. We also compared PoW with PoA for CovidBChain for various parameters such as upload and retrieval time, number of messages exchanged, and

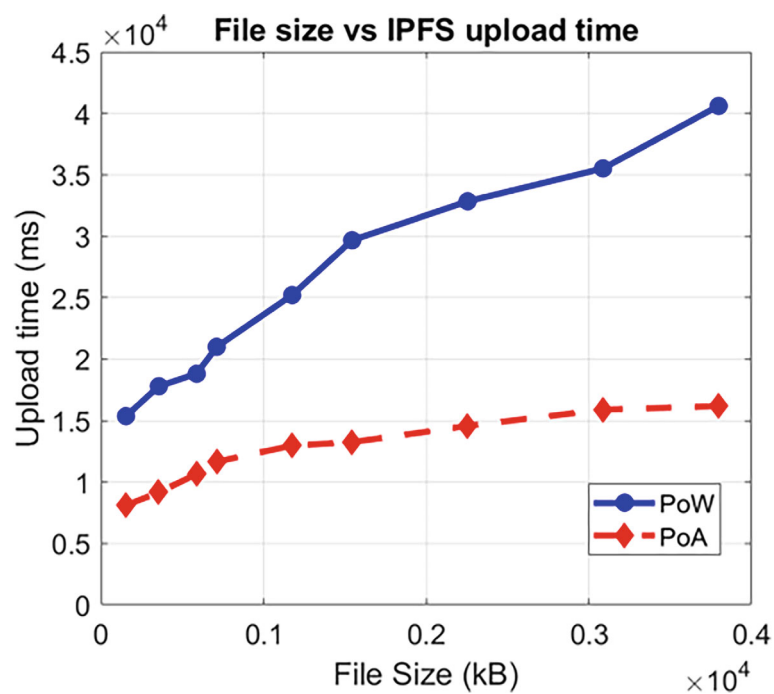


FIGURE 4 File size versus IPFS upload time

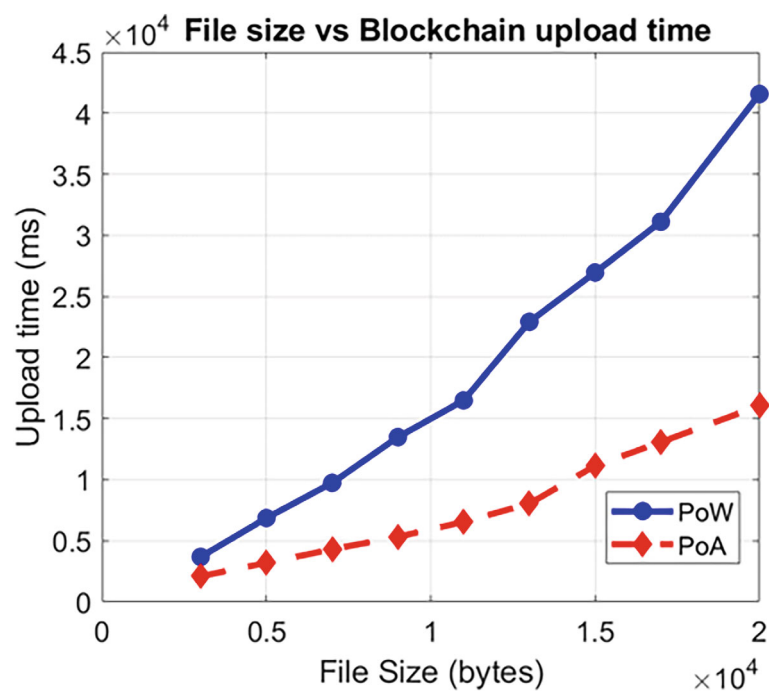


FIGURE 5 File size versus blockchain upload time

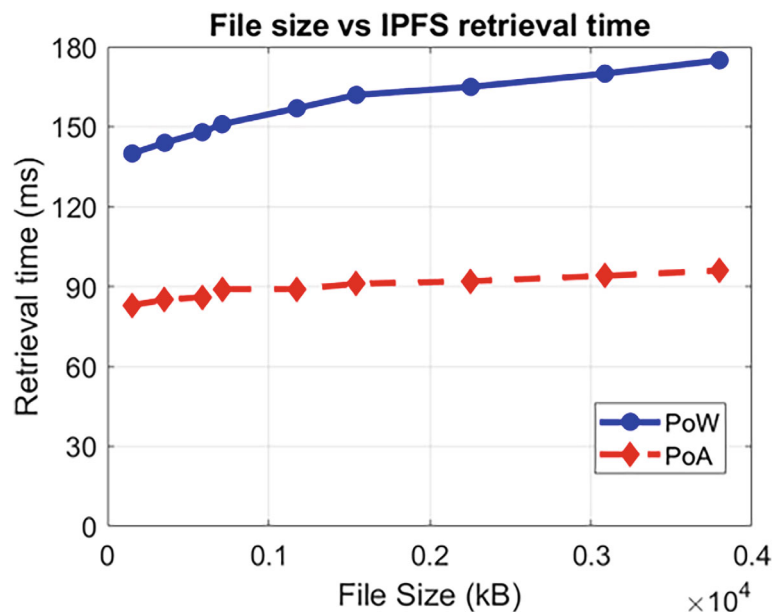


FIGURE 6 File size versus IPFS retrieval time

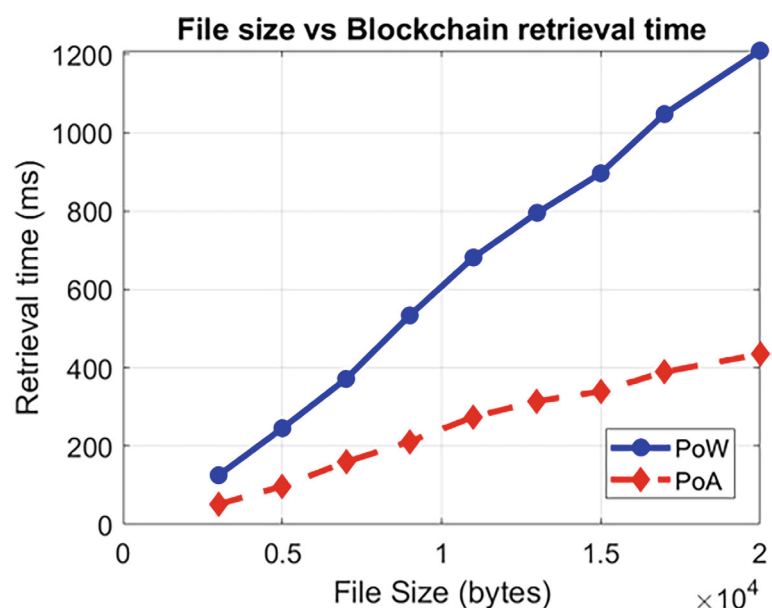


FIGURE 7 File size (kB) versus blockchain retrieval time

transaction cost and concludes that PoA outperforms PoW in every aspect except security. Table 4 presents the data structure used in the CovidBChain for patient and hospital information. The effect of file size on upload time, retrieval time, and the gas used in the case of on-chain and on/off-chain storage is measured. Table 5 shows the specifications of the experiment environment to execute the simulation.

Table 6 shows the upload and retrieval time of on-chain and off-chain storage for PoW and PoA. Figures 4 and 5 show how a file's upload time varies according to the file size using on/off-chain (combination of blockchain and IPFS) and on-chain (blockchain) storage, respectively. In Figure 4, critical information is stored on the blockchain, and noncritical data are stored on IPFS storage, that is, on/off-chain. In Figure 5, critical and noncritical information is stored on the blockchain. The upload time for on/off-chain storage showed an increment of 1.99 $\times$ (from 8100 to 16,172 ms) for PoA as compared to PoW which rises by 2.64 $\times$ (15,360 to 40,604 ms) while on-chain storage shows an increment of 7.64 $\times$ (2100, 16,060) for PoA and 11.25 $\times$ for PoW.

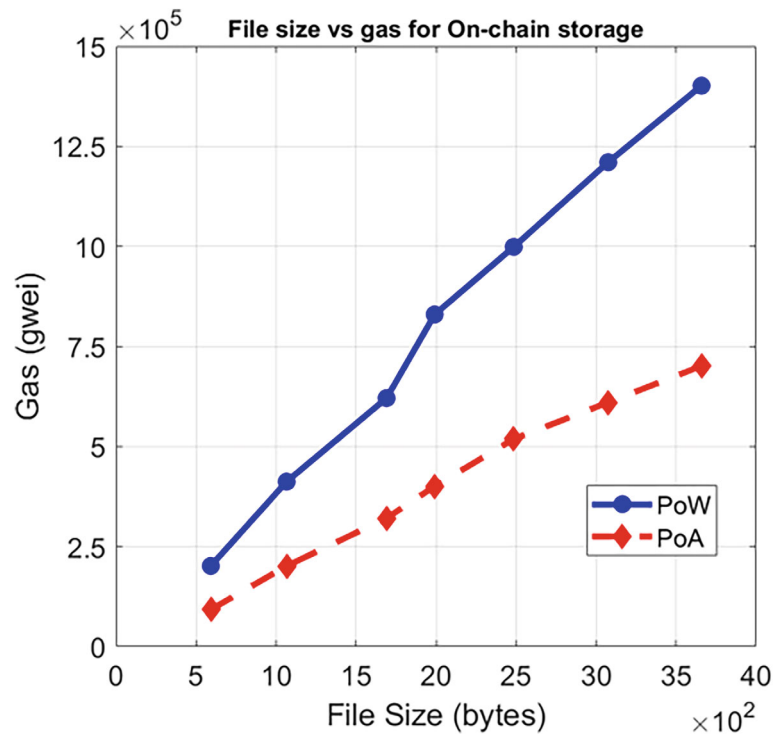


FIGURE 8 File size (kB) versus gas (gwei) for on-chain

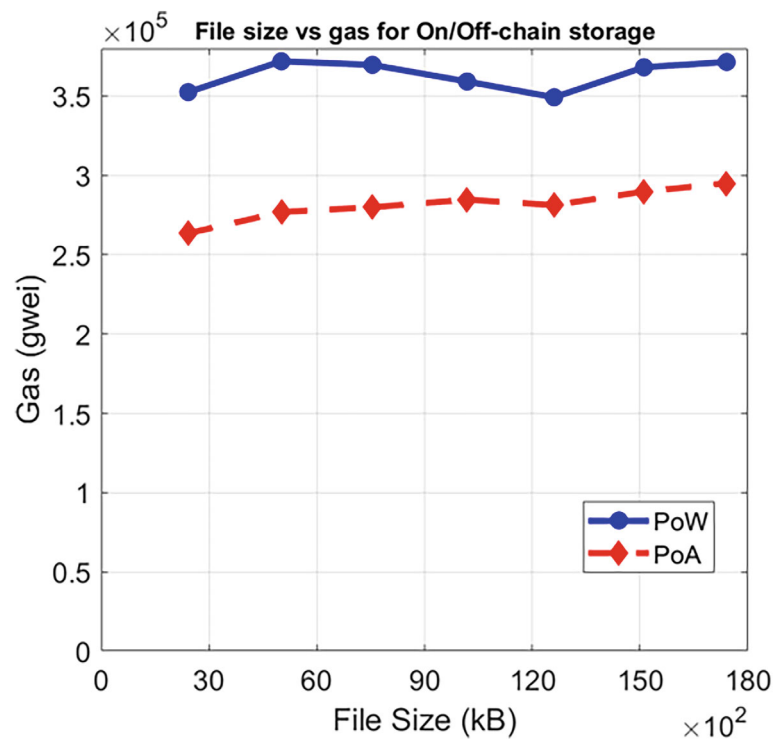


FIGURE 9 File size (kB) versus gas (gwei) for on/off-chain

TABLE 4 Data structure for patients and hospitals

Patient information	Critical data fields	Patient_Id, Name, Timestamp, Noncritical data hash
	Noncritical data fields	DoB, Weight, Covid-19 Status, Prescription, Reports
Hospital information	Critical data fields	Hospital_name, State_name, Ventilator, Oxygen cylinder, ICU beds, Timestamp, PPE status, Noncritical data hash
	Noncritical data fields	Total_cases, New_cases, Total_deaths, New_deaths, Weekly_icu_admission, Median_age, Smoke_status, Weekly_hospital_admission, Disease_history, Handwashing_facilities

TABLE 5 Experimental setup

Server specifications	Processor	Intel® Core™ i5-8200Y Processor
	Memory	8 GB RAM
	Storage	3 * 146 GB
	Operating system	Ubuntu 20.01 LTS
	Browser	Google Chrome
	Wallet	Metamask
	Blockchain environment	Ganache
	IPFS version	0.7.0

TABLE 6 File upload and retrieval time in case of on-chain and off-chain storage

S. no.	File size (kB)	On/off-chain storage				On-chain storage				
		PoW		PoA		File size (bytes)	PoW		PoA	
		Upload time (ms)	Retrieval time (ms)	Upload time (ms)	Retrieval time (ms)		Upload time (ms)	Retrieval time (ms)	Upload time (ms)	Retrieval time (ms)
1.	153	15,360	140	8100	83	3000	3694	126	2100	52
2.	355	17,789	145	9150	85	5000	7872	246	3200	98
3.	588	18,811	148	10,650	86	7000	11,752	372	4310	160
4.	711	20,998	150	11,652	89	9000	15,498	534	5330	212
5.	1175	25,211	152	12,960	89	11,000	19,504	682	6558	275
6.	1544	29,670	152	13,240	91	13,000	22,920	796	8050	315
7.	2254	32,840	155	14,565	92	15,000	26,960	897	11,150	340
8.	3087	35,517	160	15,886	94	17,000	31,118	1048	13,090	390
9.	3798	40,604	165	16,172	96	20,000	41,575	1210	16,060	435

Figures 6 and 7 show how the retrieval time of a file varies according to the file size using on/off-chain and on-chain, respectively. As the data storage increases, retrieval time also increases. The retrieval time for both the consensus mechanism, that is, (PoA and PoW), shows approximately the same incrementation in the case of on/off-chain storage, while for on-chain storage, the PoW shows an increment of $9.60\times$ (126, 1210) and $8.3\times$ (52, 435) for PoA. Figures 8 and 9 show the amount of gas consumed in on-chain and on/off-chain storage according to the file size. In on-chain storage, the gas consumed increases linearly with file size, the gas consumed is risen by $6.94\times$ for PoA and $7.56\times$ for PoW consensus, but in on/off-chain storage, gas consumed is approximately constant irrespective of data stored because only critical information is stored on the blockchain, and noncritical information is stored on the IPFS.

Figure 10 represents the number of messages exchanged between the nodes in PoW and PoA consensus, with varying network size from 10 to 100. An “N” nodes network using PoW consensus requires $N * (N - 1)$ message exchanges to achieve consensus, while PoA consensus requires only $N - (N/2 + 1)$ authorities. In PoA, the overhead of message exchange is reduced by a factor of $4.3\times$ compared to PoW.

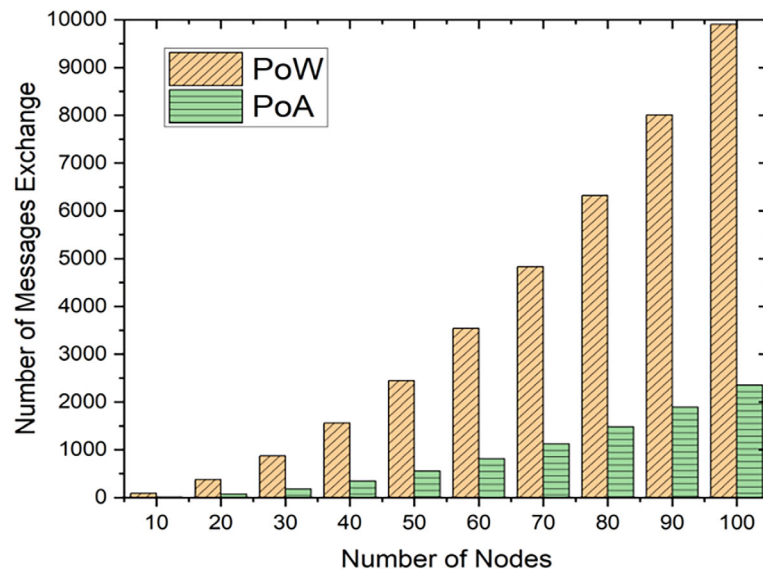


FIGURE 10 Messages exchanged versus number of nodes

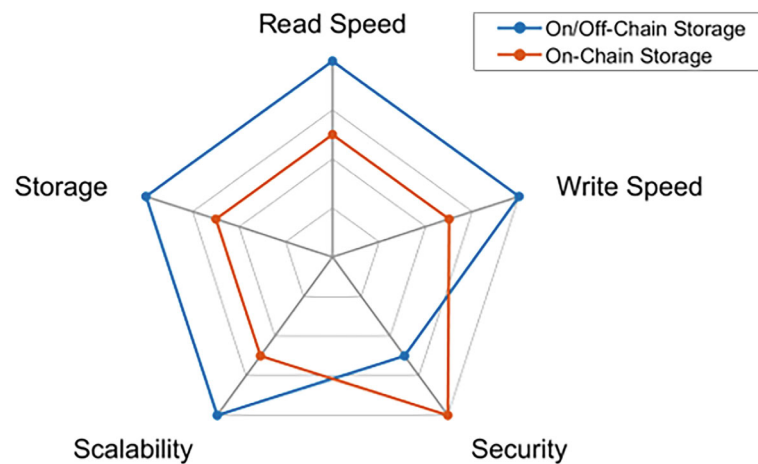


FIGURE 11 Performance comparisons of on-chain and on/off chain scheme

Figure 11 shows a performance analysis that compares the efficiency of storing all the data in the on-chain versus on/off-chain model in terms of scalability, read/write speed, storage and security. Figures 4–7 demonstrate in an application scenario such as health care data management, where a vast volume of data must be processed, the write and read performance of the segregated data storage is better than non-segregated data storage. The benefit is prominent with more data. Since the noncritical data are stored separately, the blockchain storage can be shortened. IPFS storage is not restricted by storage space and can be scaled up dynamically according to the requirement. As a result, the segregated storage scheme reduces the amount of data on the blockchain network, which has a range of benefits, including increased system scalability, fast read/write speed, and better space utilization.

The fundamental principle of blockchain is storing data in a distributed manner to ensure data integrity, immutability, transparency, and security. Our model combines the blockchain (i.e., on-chain) with the IPFS (i.e., off-chain) storage scheme. Due to storing noncritical data in an off-chain manner, the security is less than keeping all the data in the on-chain (blockchain). The on/off-chain is marginally less secure than storing data on a blockchain network but safer than conventional data storage methods.

6.2 | Security analysis

This section analyses the CovidBChain from the security aspect. Informal security analysis through mathematical equations and methods is conducted, while formal security analysis is conducted through BAN logic protocol.

TABLE 7 Security comparison between existing authentication schemes and CovidBChain

Attacks	46	47	48	49	50	CovidBChain
Session key attack	Yes	Yes	Yes	Yes	Yes	Yes
Mutual authentication	Yes	No	Yes	No	Yes	Yes
Perfect forward secrecy	No	Yes	No	Yes	Yes	Yes
Prevent replay attack	Yes	Yes	Yes	Yes	Yes	Yes
Prevent insider attack	No	No	No	No	No	Yes
Prevent impersonation attack	No	Yes	Yes	Yes	Yes	Yes

6.2.1 | Informal security analysis

CovidBChain is resistive to various attacks. Moreover, we show that it also provides secure mutual authentication and patient anonymity.

1. Impersonation attack

Suppose the identities of valid nodes are successfully captured by malicious nodes, then the chance of impersonation attack increases. If a malicious entity M_E tries to impersonate a genuine patient PT_i to access his sensitive information. The malicious entity M_E has to create a message $\langle M_1, M_{ap}, T1 \rangle$ to impersonate on behalf of (PT_i) . However, the value of M_{ap} depends on (z_i) , and the malicious entity will not be able to calculate (z_i) because to calculate (z_i) random number $(R1_{NA})$ is needed, which is generated by the network administrator. Moreover, the M_1 is encrypted with the (PT_{ski}) , that is, patient secret key. Hence, the CovidBChain is safe against impersonation attacks.

2. Session key disclosure attack

To execute a session key attack on the malicious entity, M_E must create a genuine session key $SK_{ij} = h(HID_i || MID_j || z_i || b_j)$. To generate the session key, the malicious entity M_E must know b_j . The malicious entity M_E will not get the real identities of patient PT_i and hospital HP_j as they are encrypted with random numbers p_i and HP_{skj} . Therefore, CovidBChain is resistive against the session key disclosure attacks.

3. Perfect forward secrecy

Suppose the malicious entity M_E has obtained the long-term private secret key $Net_{Adm} sk$. The malicious entity M_E will not find the previous session key, as session key $SK_{ij} = h(HID_i || PID_j || x_i || b_j)$ will not contain $(Net_{Adm} sk)$. Even if the malicious entity (M_E) gets the long-term parameter $(R1_{NA})$, the malicious entity will not be able to decode (z_i) . As (z_i) is encoded with HID_i , and HID_i is encoded with (p_i) . Therefore, CovidBChain guarantees perfect forward secrecy.

4. Replay attack

A replay attack means malicious nodes fraudulently repeat a message. Suppose a malicious entity M_E learns transmitted messages to execute the replay attack. As the messages are created, including the timestamp field, and it is a unique value. Then, the hospital (HP_j) and patient (PT_i) check that $M_{ap}^* = M_{ap}$ and $M_{amc}^* = M_{amc}$ are equal. Therefore, CovidBChain is safe against replay attacks.

5. Privileged insider attack

Suppose the malicious entity M_E is the authenticated user of the system. The insider registration information, that is, $\langle HID_i \rangle$ of a genuine user, is known to the malicious node. The values stored in the $SC_i \{W_i, X_i, Y_i, Z_i\}$ are also known to the M_E . Thus, the malicious entity cannot know $PTPWD_i$, and due to this, it will not be able to guess the valid password. Therefore, CovidBChain is safe against privileged insider attacks.

6. Anonymity

The real identities of patients and hospitals are shielded using the hash function, random numbers, and private keys. Thus, the proposed protocol hides the entities' actual identities by providing anonymity to their real identities.

7. Mutual authentication

The malicious entity (M_E) is unable to compute a correct session key. The legitimacy of the entity is checked by comparing the values of $M_{ap}^* = M_{ap}$ and $M_{amc}^* = M_{amc}$. The patient (PT_i) and hospital (HP_j) mutually authenticate if the conditions are satisfied. The proposed protocol guarantees secure mutual authentication. Table 7 shows the security comparison between the CovidBChain and existing authentication schemes.

6.3 | BAN logic analysis

Burrows et al. proposed the logic of authentication,^{44,45} which is popular in checking the correctness of authentication protocols. Formal analysis is conducted through BAN logic to validate the proposed scheme's session key agreement and secure mutual authentication between patient (PT_i) and hospital (HP_j). The BAN logic checks the source of the sent message and the scheme's freshness and trustworthiness using

TABLE 8 Basic symbols in BAN logic system

Notation	Description
A, B	Two principals (usually entity or people)
C, Z	Two statements
SK	Session key
$A \equiv C$	A believes C
$A \triangleleft C$	A sees C and received a message containing C
$A \sim C$	A once said C
$A \Rightarrow C$	A has jurisdiction or control over C
$\#(A)$	A is fresh
$A \xleftrightarrow{K} B$	A and B use the shared key K to communicate
$\{C\}_K$	C is encrypted by key K
$\langle C \rangle_Z$	C contains secret Z
$\xrightarrow{k} B$	K is the public key of B and the corresponding private key is K^{-1}

a set of rules. The notation and abbreviations used in BAN logic are listed in Table 8, followed by logic rules, goals, idealized forms, and assumptions.

BAN logic rules: The rules or logical postulates used in the BAN logic are given as follows:

Rule 1: Message meaning rule (MMR): $\frac{A \equiv A \xleftrightarrow{K} B, A \triangleleft \{C\}_K}{A \equiv B \sim C}$

If entity A believes that the secret K is shared with B and sees message C is encrypted using K, then A believes that B once said C.

Rule 2: Nonce verification rule (NVR): $\frac{A \equiv \#(C), A \equiv B \sim C}{A \equiv B \equiv C}$

If entity A believes that C is fresh and entity B once said C, then A believes that B believes C.

Rule 3: Jurisdiction rule (JR): $\frac{A \equiv B \Rightarrow C, A \equiv B \equiv C}{A \equiv C}$

If entity A believes that B has jurisdiction over C and A believes B, then A believes that C is true.

Rule 4: Freshness rule (FR): $\frac{A \equiv \#(C)}{A \equiv \#(C, Z)}$

If entity A believes that C is fresh, then A believes in the freshness of (C, Z).

Rule 5: Belief rule (BR): $\frac{A \equiv (C, Z)}{A \equiv C}$

If entity A believes (C, Z), then A believes C is true.

Goals: The goals for proving mutual authentication of the CovidBChain are defined as follows:

Goal 1: $PT_i \equiv PT_i \xleftrightarrow{SK} HP_j$

The patient (PT_i) believes that PT_i and hospital (HP_j) use the session key (SK) for communication.

Goal 2: $PT_i \equiv HP_j \equiv PT_i \xleftrightarrow{SK} HP_j$

The PT_i believes HP_j believes that PT_i and HP_j use SK for communication.

Goal 3: $HP_j \equiv PT_i \xleftrightarrow{SK} HP_j$

The HP_j believes that PT_i and HP_j use SK for communication.

Goal 4: $HP_j \equiv PT_i \equiv PT_i \xleftrightarrow{SK} HP_j$

The HP_j believes PT_i believes that PT_i and HP_j use SK for communication.

In essence, we want PT_i and HP_j to establish the session key (SK) and accept that each is the holder of a legitimate key. Furthermore, we want both parties to believe that the others believe they have established the same legitimate key.

Idealized forms:

We define the idealized forms as below.

Message 1: $PT_i \rightarrow HP_j : (z_i, PT_{ID_i}, T_1) \xrightarrow{HP_{pkj}} HP_j$

Message 2: $HP_j \rightarrow PT_i : (HP_{ID_j}, R_{HP}, T_2) z_i$

The assumptions of the BAN logic are as below:

A1: $PT_i \equiv PT_i \xleftrightarrow{z_i} HP_j$

$$A2: HP_j | \equiv \#(HP_{pkj})$$

$$A3: PT_i | \equiv \#(PT_{pki})$$

$$A4: PT_i | \equiv HP_j \Rightarrow (PT_i \xleftrightarrow{SK} HP_j)$$

$$A5: HP_j | \equiv PT_i \Rightarrow (PT_i \xleftrightarrow{SK} HP_j)$$

$$A6: HP_j | \equiv \#(z_i)$$

$$A7: HP_j | \equiv \#(T_1)$$

$$A8: PT_i | \equiv \#(T_2)$$

BAN logic proof: We perform the BAN logic analysis. The detailed steps are as follows.

Step 1: S1 is obtained from Message 1

$$S1 : HP_j \triangleleft (z_i, PT_{IDi}, T_1) \xrightarrow{HP_{pkj}} HP_j$$

Step 2: S2 is obtained by applying the MMR using S1 and A2

$$S2 : HP_j | \equiv PT_i | \sim (z_i, PT_{IDi}, T_1)$$

Step 3: S3 is obtained by applying the FR using S2 and A6

$$S3 : HP_j | \equiv \#(z_i, PT_{IDi}, T_1)$$

Step 4: S4 is obtained by applying the NVR using S2 and S3.

$$S4 : HP_j | \equiv PT_i | \equiv (z_i, PT_{IDi}, T_1)$$

Step 5: S5 is obtained from S4, A7 and the BR.

$$S5 : HP_j | \equiv PT_i | \equiv (z_i, PT_{IDi})$$

Step 6: Because of the session key $SK = h(PT_{IDi} || HP_{IDj} || z_i || R_{HP})$, from S5 and A3

$$S6 : HP_j | \equiv PT_i | \equiv (PT_i \xleftrightarrow{SK} HP_j)$$

Step 7: S7 is obtained by applying the JR using A5 and S6.

$$S7 : HP_j | \equiv (PT_i \xleftrightarrow{SK} HP_j)$$

Step 8: S8 is obtained from Message 2

$$S8 : PT_i \triangleleft (HP_{IDj}, R_{HP}, T_2)_{z_i}$$

Step 9: S9 is obtained by applying the MMR using A1 and S8.

$$S9 : PT_i | \equiv HP_j | \sim (HP_{IDj}, R_{HP}, T_2)_{z_i}$$

Step 10: S10 is obtained by applying the FR using A3 and S9.

$$S10 : PT_i | \equiv \#(HP_{IDj}, R_{HP}, T_2)_{z_i}$$

Step 11: S11 is obtained by applying the NVR using S9 and S8

$$S11 : PT_i | \equiv HP_j | \equiv (HP_{IDj}, R_{HP}, T_2)_{z_i}$$

Step 12: S12 is obtained by applying the BR using A8 and S11.

$$S12 : PT_i | \equiv HP_j | \equiv (HP_{IDj}, R_{HP})_{z_i}$$

Step 13: S13 is obtained from A6, S12, and the session key $SK = h(PT_{IDi} || HP_{IDj} || z_i || R_{HP})$.

$$S13 : PT_i | \equiv HP_j | \equiv (PT_i \xleftrightarrow{SK} HP_j)$$

Step 14: S14 is obtained by applying the JR using A4 and S13.

$$S14 : HP_j | \equiv (PT_i \xleftrightarrow{SK} HP_j)$$

Therefore, goals 1–4 clearly show that the proposed protocol provides secure mutual authentication between PT_i and HP_j .

6.3.1 | Comparison between the traditional and CovidBChain healthcare system

Table 9 presents a qualitative analysis of the CovidBChain and the traditional healthcare system.

7 | DISCUSSION AND FINDINGS

The use case scenario of the current research is presented in this section. The greedy healthcare system stakeholders such as hospital staff, doctors, and owners seek opportunities in the Covid-19 disaster to earn money. As discussed in Section 1, many hospitals have made money by charging much higher than the actual cost of treating Covid-19 patients. Hospitals also create chaos amongst people by publishing wrong information on their portals. When patients reach the particular hospital, they will show that their amenities are already occupied. In that case, hospitals increase the charges for treatment as demand for specific equipment increases. In such a scenario, the patient will pay more in the compulsion to get the desired treatment. Thus, if the hospitals of a particular city are put on the blockchain network. Then the information which every hospital is showing inherently has all the properties of blockchain, that is, immutability, integrity, and transparency, which will

TABLE 9 Qualitative analysis of the CovidBChain and traditional healthcare system

Features	Traditional healthcare system	CovidBChain healthcare system
Confidentiality	End to end encryption is provided to the transmitted data	Same level of confidentiality
Immutability	Databases are vulnerable to malicious manipulation	Once a block is added, it is computationally infeasible to mutate
Traceability	Information may alter which hampers traceability	Transactions are traceable to their provenance guarantees immutability
Availability	Backups must be managed manually to tackle the failure	All nodes have an exact copy of the blockchain, which provides high fault tolerance
Privacy	Data are transmitted in encrypted form, which can be traced back to the data owner	Anonymous references protect privacy and discard any possibility of an association
Speed	Transactions are mainly depending on the network transfer rate	A negligible delay occurs due to the validation of block

increase the population's reliability for the hospital in the blockchain network compared to the hospital, not on the blockchain network. In the long run or even in the current situation, blockchain network hospitals' reliability and popularity are more than those not part of the blockchain network.

8 | CONCLUSIONS

Despite the advent of EHRs, the healthcare domain is still lagging. The current research shows how government agencies and hospitals manipulate various hospital amenities and patient Covid-19 information to gain benefit. This article discusses how CovidBChain preserves the integrity of healthcare data through fine-grained access and controlled authentication. It aims to create an information system that carries Covid-19 data such as ventilator take/release time, Covid-19 beds left/occupied, oxygen cylinder, ICU occupancy, and so forth. Each such operation is integrated into the blockchain as a transaction. The blockchain provides a tamper-proof way to conduct transactions without a central authority. Thus, EHRs, medical equipment, and amenities information cannot be modified after the corresponding transaction is recorded into the blockchain. The architecture of a multi-layered blockchain-based CovidBChain system is presented in the current study. CovidBChain defines four distinctive layers: presentation, service, blockchain, and data. It elaborates on various stakeholders, that is, patients, medical centers, doctors, insurers, and their use case scenarios. Security analysis through BAN logic and results with the help of upload and query response time, number of messages exchanged, and transaction cost are discussed. The upload and retrieval time for off-chain storage showed an increment of 1.99× and 1.15× for PoA as compared to PoW, which rises by 2.64× and 1.17×, while on-chain storage shows an increment of 7.64× and 8.3× for PoA and 11.25× and 9.6× for PoW. The PoA also reduces the message exchange by a factor of 4.3× as compared to PoW. The traditional system with CovidBChain is also compared based on various parameters such as confidentiality, immutability, traceability, availability, privacy, and speed. Thus, CovidBChain gives a real-time availability of the essential hospital amenities required to tackle Covid-19 and helps in the state-wide transfer of crucial medical equipment from one state to another to fulfill the dynamic requirements.

CONFLICT OF INTEREST

The authors declare no potential conflict of interest.

DATA AVAILABILITY STATEMENT

Data sharing is not applicable to this article as no new data were created or analyzed in this study.

ORCID

Shelly Sachdeva  <https://orcid.org/0000-0003-4088-1271>

REFERENCES

1. Karthiyayini R, Shenbagavadivu N. Retinal image analysis for ocular disease prediction using rule mining algorithms. *Interdiscipl Sci Comput Life Sci*. 2021;13(3):451-462.
2. Sridhar C, Bhat S, Acharya UR, Adeli H, Bairy GM. Diagnosis of attention deficit hyperactivity disorder using imaging and signal processing techniques. *Comput Biol Med*. 2017;88:93-99.

3. Zeng A, Rong H, Pan D, et al. Discovery of genetic biomarkers for Alzheimer's disease using adaptive convolutional neural networks ensemble and genome-wide association studies. *Interdiscipl Sci Comput Life Sci*. 2021;13(4):787-800.
4. Firth S. Public health experts worried about possible COVID-19 data manipulation; September 24, 2020.
5. Panneerselvam A. The uncertainties over COVID-19 numbers; August 17, 2020.
6. Koch C, Okamura K. Benford's law and COVID-19 reporting. *Econ Lett*. 2020;196:109573.
7. Mansukh A. Profit in times of covid 19 is it time to take over private hospitals; August 27, 2020.
8. Kalpagge K. Mismatch in Delhi govts data continues corona app health bulletin show different figures. *Sepsis* 11, 2020.
9. Bassi A, Arfin S, John O, Jha V. An overview of mobile applications (apps) to support the coronavirus disease 2019 response in India. *Indian J Med Res*. 2020;151(5):468.
10. Stevens MP, Doll M, Pryor R, Godbout E, Cooper K, Bearman G. Impact of COVID-19 on traditional healthcare-associated infection prevention efforts. *Infect Control Hosp Epidemiol*. 2020;41(8):946-947.
11. Diseases TLI. The COVID-19 infodemic. *Lancet Infect Dis*. 2020;20(8):875.
12. Li W, Wang Y, Li J, Au MH. Toward a blockchain-based framework for challenge-based collaborative intrusion detection. *Int J Inf Secur*. 2021;20(2):127-139.
13. Wu H, Dwivedi AD, Srivastava G. Security and privacy of patient information in medical systems based on blockchain technology. *ACM Trans Multimed Comput Commun Appl (TOMM)*. 2021;17(2s):1-17.
14. Zhang R, Xue R, Liu L. Security and privacy on blockchain. *ACM Comput Surv (CSUR)*. 2019;52(3):1-34.
15. Xu H, Zhang L, Onireti O, Fang Y, Buchanan WJ, Imran MA. BeepTrace: blockchain-enabled privacy-preserving contact tracing for COVID-19 pandemic and beyond. *IEEE Internet Things J*. 2020;8(5):3915-3929.
16. Kumar R, Khan AA, Kumar J, et al. Blockchain-federated-learning and deep learning models for COVID-19 detection using CT imaging. *IEEE Sensors J*. 2021;21(14):16301-16314.
17. Arifeen MM, Al Mamun A, Kaiser MS, Mahmud M. Blockchain-enable contact tracing for preserving user privacy during COVID-19 outbreak; 2020.
18. Marbough D, Abbasi T, Maasmi F, et al. Blockchain for COVID-19: review, opportunities, and a trusted tracking system. *Arab J Sci Eng*. 2020;45(12):9895-9911.
19. Chang MC, Park D. How can blockchain help people in the event of pandemics such as the COVID-19. *J Med Syst*. 2020;44(5):1-2.
20. Torky M, Hassanien AE. COVID-19 blockchain framework: innovative approach. arXiv preprint arXiv:2004.06081, 2020.
21. Bansal A, Garg C, Padappayil RP. Optimizing the implementation of COVID-19 eimmunity certificates using blockchain. *J Med Syst*. 2020;44(9):1-2.
22. Çalik E, Kaya H, Çelebi FV. A novel method to ensure the security of the shared medical data using smart contracts: organ transplantation sample. *Concurr Comput Pract Exp*. 2022;34(9):e6752.
23. Mashamba-Thompson TP, Crayton ED. Blockchain and artificial intelligence technology for novel coronavirus disease 2019 self-testing, 2020.
24. Kaur J, Rani R, Kalra N. Blockchain-based framework for secured storage, sharing, and querying of electronic healthcare records. *Concurr Comput Pract Exp*. 2021;33(20):e6369.
25. Verma A, Bhattacharya P, Zuhair M, Tanwar S, Kumar N. VacoChain: blockchain-based 5g-assisted UAV vaccine distribution scheme for future pandemics. *IEEE J Biomed Health Inform*. 2021;26(5):1997-2007.
26. Lee TF, Li HZ, Hsieh YP. A blockchain-based medical data preservation scheme for telecare medical information systems. *Int J Inf Secur*. 2021;20(4):589-601.
27. Ahmad RW, Salah K, Jayaraman R, Yaqoob I, Omar M, Ellahham S. Blockchain-based forward supply chain and waste management for COVID-19 medical equipment and supplies. *IEEE Access*. 2021;9:44905-44927.
28. Omar IA, Debe M, Jayaraman R, Salah K, Omar M, Arshad J. Blockchain-based supply chain traceability for COVID-19 personal protective equipment. *Comput Ind Eng*. 2022;167:107995.
29. Kumar M, Chand S. MedHypChain: a patient-centered interoperability hyperledger-based medical healthcare system: regulation in COVID-19 pandemic. *J Netw Comput Appl*. 2021;179:102975.
30. Kumar M, Rani R. SAI-BA-IoMT: secure AI-based blockchain-assisted internet of medical things tool to moderate the outbreak of COVID-19 crisis. arXiv preprint arXiv:2108.09539, 2021.
31. Samuel O, Omojo A, Onuja A, et al. IoMT: a COVID-19 healthcare system driven by federated learning and blockchain. *IEEE J Biomed Health Inform*. 2022.
32. Liu M, Zhang Z, Chai W, Wang B. Privacy-preserving COVID-19 contact tracing solution based on blockchain. *Comput Stand Interf*. 2023;83:103643.
33. Dolev D, Yao A. On the security of public key protocols. *IEEE Trans Inf Theory*. 1983;29(2):198-208.
34. Canetti R, Krawczyk H. Analysis of key-exchange protocols and their use for building secure channels; 2001:453-474; Springer.
35. Canetti R, Krawczyk H. Universally composable notions of key exchange and secure channels; 2002:337-351; Springer.
36. Madhusudhan R, Mittal R. Dynamic ID-based remote user password authentication schemes using smart cards: a review. *J Netw Comput Appl*. 2012;35(4):1235-1248.
37. Limbasiya T, Doshi N. An analytical study of biometric based remote user authentication schemes using smart cards. *Comput Electr Eng*. 2017;59:305-321.
38. Kocher P, Jaffe J, Jun B. Differential power analysis; 1999:388-397; Springer.
39. Wood G. Ethereum: a secure decentralised generalised transaction ledger. *Ethereum Project Yellow Paper*. 2014;151:1-32.
40. Ropsten testnet explorer. Accessed August 3, 2021. <https://ropsten.etherscan.io>
41. Goerli test network. Accessed June 29, 2021. <https://goerli.etherscan.io>
42. Coronavirus source data - our world in data. Accessed June 25, 2021. <https://ourworldindata.org/coronavirus-source-data>
43. Tuberculosis chest X-ray image data sets. Accessed June 20, 2021. <https://lhncbc.nlm.nih.gov/publication/pub9356>
44. Burrows M, Abadi M, Needham RM. A logic of authentication. *Proc Royal Soc Lond A Math Phys Sci*. 1989;426(1871):233-271.
45. Lee J, Yu S, Park K, Park Y, Park Y. Secure three-factor authentication protocol for multi-gateway IoT environments. *Sensors*. 2019;19(10):2358.
46. Tsai JL, Lo NW. A privacy-aware authentication scheme for distributed mobile cloud computing services. *IEEE Syst J*. 2015;9(3):805-815.
47. Yang TC, Lo NW, Liaw HT, Wu WC. A secure smart card authentication and authorization framework using in multimedia cloud. *Multimed Tools Appl*. 2017;76(9):11715-11737.

48. Shajina A, Varalakshmi P. A novel dual authentication protocol (DAP) for multi-owners in cloud computing. *Clust Comput*. 2017;20(1):507-523.
49. Anakath A, Rajakumar S, Ambika S. Privacy preserving multi factor authentication using trust management. *Clust Comput*. 2019;22(5):10817-10823.
50. Chaudhry SA, Kim IL, Rho S, Farash MS, Shon T. An improved anonymous authentication scheme for distributed mobile cloud computing services. *Clust Comput*. 2019;22(1):1595-1609.

How to cite this article: Pawar V, Sachdeva S. CovidBChain: Framework for access-control, authentication, and integrity of Covid-19 data. *Concurrency Computat Pract Exper*. 2022;34(28):e7397. doi: 10.1002/cpe.7397